

CONTENTPULSE

AI Social Publishing & Human-Approval Engine

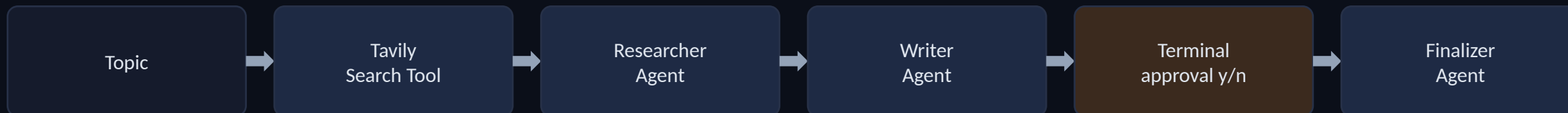
Architecture, Technology Stack & End-to-End Code Walkthrough

What ContentPulse Does

- A pipeline that turns a topic into a published social post — with a human always in the loop.
- Flow: Topic → Live web research → AI-drafted copy → Human review & edit → Publish
- Two parallel implementations of the same idea:
 - A terminal / Google ADK agent pipeline (agent.py) — approval via a blocking prompt
 - A FastAPI web dashboard (server.py + index.html + app.js) — approval via a browser UI
- Current publish targets: LinkedIn and Facebook (Instagram & Twitter/X were removed by design).

Two Execution Paths, One Concept

CLI PATH (adk run)



WEB PATH (FastAPI dashboard)



Shared external services

Tavily Search API • OpenRouter (LiteLLM proxy → openai/gpt-4o-mini by default) • LinkedIn Posts API • Meta Graph API (Facebook Page Feed)

Technology Stack



Backend

- Python 3
- FastAPI + Uvicorn
- Pydantic (schemas)
- python-dotenv
- requests



AI / Agents

- Google ADK (Agent Development Kit)
- LiteLLM (OpenRouter proxy)
- OpenRouter — openai/gpt-4o-mini
- Tavily Search API



Publishing

- LinkedIn Posts (UGC) API
- Meta Graph API — Facebook Page Feed
- Simulated “sandbox” mode when no live credentials



Frontend

- Vanilla HTML5 / CSS3
- Vanilla JavaScript (Fetch API)
- No build step, no framework
- In-memory Python list for post history

End-to-End Request Flow (Web Path)

- 1. POST /api/pipeline/generate — runs Tavily research, then calls the LLM to draft a post; returns draft + is_fallback flag
- 2. Reviewer reads the draft in the browser; a red banner appears if it's a fallback template, not a real AI draft
- 3. POST /api/pipeline/refine (optional, repeatable) — “Punchier” / “Add emojis” / custom instruction; fails safe, never silently overwrites the draft
- 4. Reviewer clicks Approve — one channel, or “Approve & One-Click Publish” for all selected channels
- 5. POST /api/publish/linkedin and/or /api/publish/facebook — live API call if credentials exist, otherwise an explicitly labeled simulated_sandbox response
- 6. Result recorded in POST_HISTORY and reflected in the UI (“Published” vs “Simulated (sandbox)”)

CLI Pipeline — Setup & Model Wiring

content_pipeline/agent.py

```
OPENROUTER_API_KEY = os.environ.get(
    "OPENROUTER_API_KEY")
OPENROUTER_MODEL_NAME = os.environ.get(
    "OPENROUTER_MODEL", "openai/gpt-4o-mini")

def _openrouter_model(tool_choice=None):
    options = {
        "api_key": OPENROUTER_API_KEY,
        "api_base": OPENROUTER_API_BASE,
        "num_retries": 2,
    }
    return LiteLlm(
        model=f"openrouter/{OPENROUTER_MODEL_NAME}",
        **options,
    )
```

- Runs under Google ADK's SequentialAgent framework, invoked with `adk run content\_pipeline`
- Every agent talks to OpenRouter through LiteLLM's model-agnostic wrapper — changing OPENROUTER_MODEL swaps the model with no code change
- Secrets load once via load_dotenv() from content_pipeline/.env
- A separate implementation from server.py's call_llm() — they share no code

CLI Pipeline — The Research Tool

content_pipeline/agent.py

```
def tavily_search(query: str) -> str:
    if not TAVILY_API_KEY:
        raise ValueError(
            "TAVILY_API_KEY is missing..."
        )

    response = requests.post(
        TAVILY_API_URL,
        json={
            "api_key": TAVILY_API_KEY,
            "query": query,
            "max_results": 5,
            "include_answer": True,
        },
        timeout=30,
    )
    ...

search_tool = FunctionTool(tavily_search)
```

- A plain Python function, wrapped as an ADK FunctionTool so the LLM can call it autonomously
- Unlike server.py, this path raises loudly on a missing key rather than silently substituting mock data
- Formats the Tavily answer plus top 5 results into a compact text block the researcher agent reads
- Attached only to researcher_agent via its tools=[search_tool] argument

The Three-Agent Pipeline

1

researcher_agent

Calls tavily_search, condenses findings into 2-3 factual notes

output_key:
research_notes

2

writer_agent

Writes a 2-3 line punchy post from the notes; triggers the approval callback when done

output_key:
draft

3

finalizer_agent

Echoes back {final_post} — the approved (or rejected) text — as the pipeline's output

output_key:
final_post

CLI Pipeline — Human Approval & Orchestration

content_pipeline/agent.py

```
async def _prompt_for_approval_after_draft(
    callback_context) -> None:
    draft = callback_context.state.get(
        "draft", "").strip()
    ...
    answer = input(
        f"\nPost this?\n\n{draft}\n\n"
        "Publish? (y/n): ")
    if answer == "y":
        state["approval_status"] = "approved"
        state["final_post"] = draft
    else:
        state["approval_status"] = "rejected"

content_pipeline_flow = SequentialAgent(
    sub_agents=[researcher_agent,
                writer_agent, finalizer_agent])
```

- Registered as writer_agent's after_agent_callback — fires the instant the draft is written
- Blocking input() requires a real terminal, so this pipeline is NOT compatible with `adk web`
- SequentialAgent chains all three agents in fixed order — no branching, no retry loop on rejection
- root_agent = content_pipeline_flow is the entry point ADK's CLI runner looks for

Web Backend — Setup & Config Store

server.py

```
CONFIG = {
    "OPENROUTER_API_KEY": os.environ.get(
        "OPENROUTER_API_KEY", ""
    ),
    "TAVILY_API_KEY": os.environ.get(
        "TAVILY_API_KEY", ""
    ),
    "LINKEDIN_ACCESS_TOKEN": ...,
    "FACEBOOK_PAGE_ACCESS_TOKEN": ...,
}

POST_HISTORY: List[Dict[str, Any]] = []

app = FastAPI(
    title="ContentPulse AI & Approval Engine"
)
app.mount("/static",
    StaticFiles(directory=str(static_dir)))
```

- CONFIG is a live, mutable dict — POST /api/config updates it at runtime, no server restart needed
- POST_HISTORY is a simple in-process list; no database, so history resets on restart
- Pydantic models (GenerateRequest, RefineRequest, PublishLinkedInRequest, PublishFacebookRequest, ConfigUpdateRequest) validate every request body
- Serves index.html at GET / and the static/ folder directly — the whole frontend ships from this one FastAPI process

Web Backend — AI Generation & Honest Fallbacks

server.py

```
def call_llm(prompt, system_prompt)
    -> tuple[str, bool]:
    if not api_key:
        # Not a generated draft — flag it.
        return (NO_KEY_FALLBACK_TEXT, True)
    try:
        resp = requests.post(
            f"{base_url}/chat/completions",
            headers=headers, json=payload)
        resp.raise_for_status()
        return (
            data["choices"][0]["message"]
            ["content"].strip(),
            False)
    except Exception:
        return (fallback_text, True)
```

- Returns (text, is_fallback) — the caller always knows whether this is a real AI draft or a placeholder
- generate_post() threads is_fallback into the API response; the UI shows a red “not a real draft” banner when true
- refine_post() goes further: on fallback it raises HTTP 502 and leaves the user's draft untouched, instead of silently overwriting it
- Tavily failures are tagged research_failed / not_configured so a search error is never fed to the LLM disguised as a fact

Web Backend — Publishing, Live vs. Sandbox

server.py

```
@app.post("/api/publish/linkedin")
async def publish_linkedin(
    req: PublishLinkedInRequest):
    token = CONFIG.get(
        "LINKEDIN_ACCESS_TOKEN")
    if not token:
        return {"status": "not_configured"}
    resp = requests.post(LI_API, ...)
    if resp.status_code in (200, 201):
        post_id = resp.headers.get(
            "x-restli-id")
        return {
            "status": "success",
            "mode": "live",
            "url": f"...{post_id}"
                if post_id else None}
```

- Every publish response carries an explicit mode: "live" or "simulated_sandbox" — the frontend renders these differently
- A simulated response never fabricates a real-looking post URL; a missing real ID means url: null, never a guessed link
- Facebook has no simulate path at all — no credentials means an honest not_configured refusal, no dry run
- _record_publish_result() logs every attempt (live or simulated) into POST_HISTORY against the originating post_id

What Changed During Hardening

- Removed Twitter/X and Instagram entirely — backend endpoints, OAuth flow, frontend tabs, mockups, and settings fields, front to back.
- No more fake-as-real output:
 - AI fallback drafts are flagged (`is_fallback`) and shown with a warning banner instead of looking like a real draft
 - Simulated publishes are labeled “Simulated (sandbox)”, never “Published”
 - Research failures are excluded from the LLM prompt instead of being fed in as fabricated facts
- Fixed two reflected-XSS points (research link URLs; the since-removed OAuth callback)
- Fixed a naming bug (`publishToFacebook` vs `publishFacebook`) that silently broke the one-click multi-publish flow
- Cleaned a corrupted, UTF-16-encoded `.gitignore` and closed the root-level `.env` gap

Where the Project Stands

In scope today

- LinkedIn + Facebook publishing (live & sandbox)
- AI draft + refine loop with honest fallback labeling
- CLI (ADK) and web (FastAPI) pipelines, side by side
- Human approval gate on every path

Known open items

- No database — POST_HISTORY resets on restart
- requirements.txt missing explicit direct deps
- server.py could be split (config / AI / publish / routes)
- Publish history isn't rendered in the UI yet